reply-bridge Plugin Tool

reply-bridge Plugin Tool

1. Course Content
   - Course Overview
   - Learning Objectives
3. Introduction to reply-bridge Plugin
4. Demo Case
5. Source Code Analysis
   - 5.1 Plugin Entry and Hook Registration
   - 5.2 TTS Tag Extraction and Deduplication Mechanism
   - 5.3 TTS Server

1. Course Content

Course Overview

- reply-bridge plugin is a speech synthesis bridging plugin for OpenClaw. It listens to AI response messages from OpenClaw, parses the `<tts>...</tts>` speech tags, automatically calls the TTS tool on the MCP server for speech synthesis, and plays the audio through a USB audio device.
- When used with the robot-control plugin, it enables voice dialogue interaction with the robot, allowing the robot to "speak".

Learning Objectives

1. Understand the functionality of the reply-bridge plugin
2. Master plugin installation, viewing, and basic management operations
3. Understand the working principle of the plugin using MCP protocol to invoke TTS speech synthesis
4. Be able to use `<tts>` tags to implement voice broadcast functionality in OpenClaw interaction scenarios

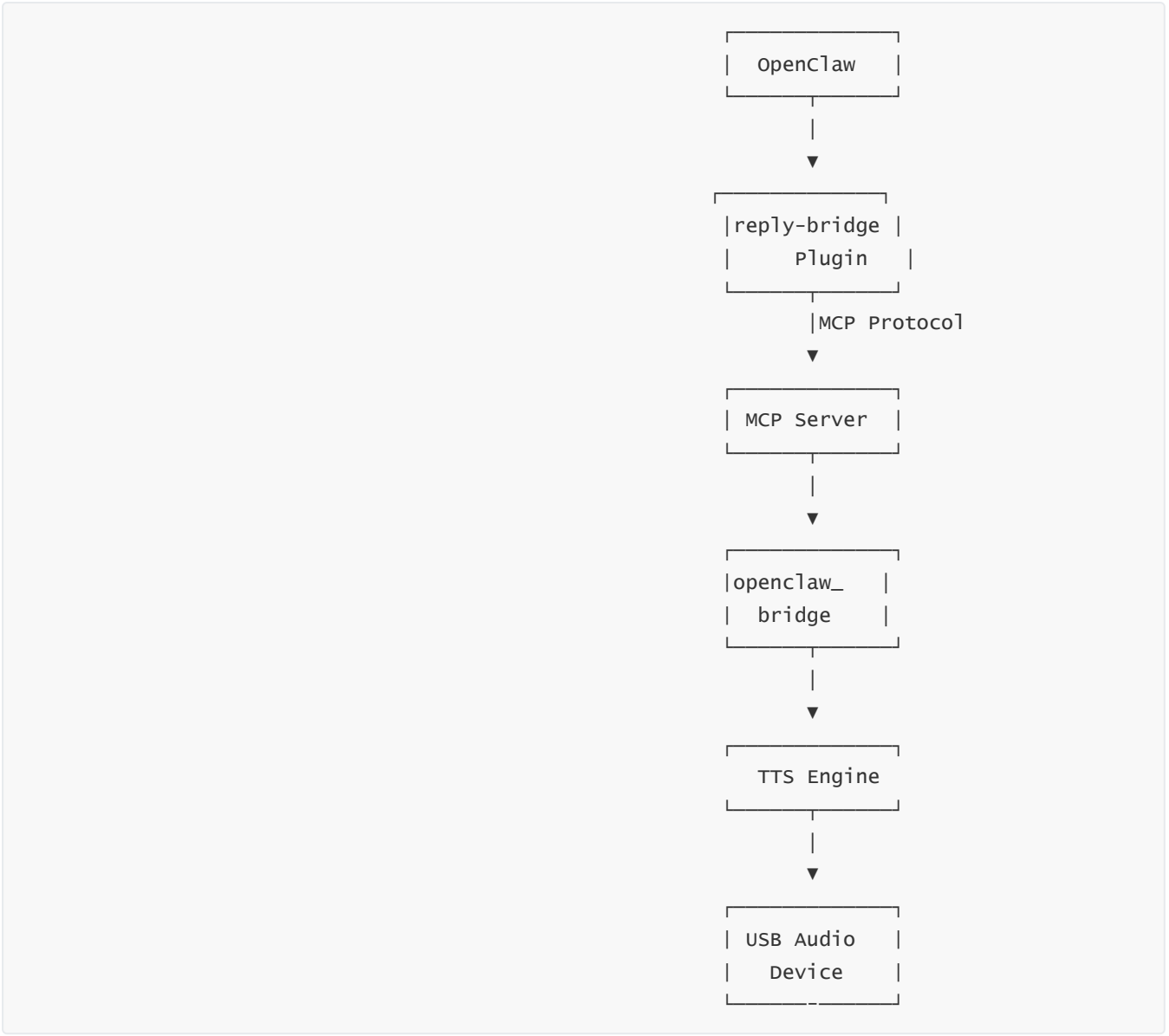
3. Introduction to reply-bridge Plugin

reply-bridge plugin is a speech synthesis bridging plugin adapted for OpenClaw vehicle-side. Its working principle is as follows:

- **Listening Mechanism**: The plugin registers two hooks: `before_message_write` and `message_sending`, to listen for `<tts>...</tts>` tags in AI response messages
- **Text Extraction**: When an AI response contains `<tts>label text</tts>`, the plugin automatically extracts the text content within the tags
- **Speech Synthesis**: Through a built-in MCP client, it calls the `TTS` tool on the MCP server to convert text to speech
- **Audio Playback**: The MCP server calls the TTS engine through the `openclaw_bridge` node, and finally plays the speech through a USB audio device

- **Deduplication Mechanism**: The plugin has a session-level deduplication mechanism built in; the same TTS text will not be repeated in the session to avoid voice redundancy

The plugin communication architecture is as follows:



4. Demo Case

- Refer to the preceding chapter [OpenClaw Interaction Methods] — [OpenClaw Voice Dialogue Interaction] section for dialogue functionality; this section only explains the implementation principle of the voice dialogue function

5. Source Code Analysis

5.1 Plugin Entry and Hook Registration

Source code directory:

```
$HOME/.openclaw/extensions/reply-bridge/index.ts
```

The plugin source code is located in `index.ts` under the plugin directory. It uses the `registerReplyBridge` function as the entry point and registers event listener hooks using the OpenClaw Plugin API. The core code is as follows:

```
export default function registerReplyBridge(api: OpenClawPluginApi) {
  // Hook: before_message_write — triggered when each message is written to session history
  api.on("before_message_write", (event, ctx) => {
    const message = event.message as AgentMessage;
    const sessionId = ctx.sessionId;

    // Only process messages with assistant role
    if (message.role !== "assistant") return;

    const text = extractTextFromAgentMessage(message);
    if (!text) return;

    const ttsTexts = extractTtsTags(text);
    if (ttsTexts.length === 0) return;

    api.logger.info(`[reply-bridge] extracted ${ttsTexts.length} TTS segment(s)`);
    sendTtsTextsFireAndForget(ttsTexts, sessionId, (msg) => api.logger.info(msg));
  });

  api.logger.info("reply-bridge: Plugin registered (hooks: before_message_write,
message_sending)");
}
```

Code Description:

1. **Registration Entry**: `export default function registerReplyBridge(api)` is the plugin's default export function. OpenClaw automatically calls it when loading the plugin, passing in the `OpenClawPluginApi` instance
2. `api.on("before_message_write")`: Registers a pre-message-write hook that triggers when the AI generates a response and writes it to session history—this is the main channel for TTS processing
3. **Role Filtering**: `message.role !== "assistant"` ensures only AI assistant response messages are processed, ignoring user messages
4. **Extraction Flow**: `extractTextFromAgentMessage` → `extractTtsTags` → `sendTtsTextsFireAndForget` —three steps complete the full chain from message text to TTS playback
5. **Fire-and-Forget Mode**: TTS calls use a fire-and-forget pattern that does not block the message write process, ensuring a smooth user experience

Key Function Description:

- `extractTextFromAgentMessage(message)`: Extracts plain text content from an AgentMessage object, handling possible multimodal content (text, images, etc.) in the message
- `extractTtsTags(text)`: Parses all content within `<tts>...</tts>` tags from the text, returning a list of TTS text fragments
- `sendTtsTextsFireAndForget(ttsTexts, sessionId, logger)`: Asynchronously sends the extracted TTS text to the MCP server for speech synthesis using a fire-and-forget pattern without blocking the main flow
- `ctx.sessionId`: Current session ID, used for session-level TTS deduplication to ensure the same text is not repeatedly played in the same session

5.2 TTS Tag Extraction and Deduplication Mechanism

The code for extracting `<tts>` tag content from AI response text is as follows:

```
/**
 * Extract plain text fragments wrapped in <tts>...</tts> tags
 * Uses indexOf instead of regex to avoid boundary issues with pattern matching
 */
function extractTtsTags(content: string): string[] {
  const matches: string[] = [];
  const openTag = "<tts>";
  const closeTag = "</tts>";
  let start = 0;

  while (true) {
    const openIdx = content.indexOf(openTag, start);
    if (openIdx === -1) break;
    const closeIdx = content.indexOf(closeTag, openIdx + openTag.length);
    if (closeIdx === -1) break;
    const text = content.slice(openIdx + openTag.length, closeIdx).trim();
    if (text) matches.push(text);
    start = closeIdx + closeTag.length;
  }

  return matches;
}
```

Code Description:

1. `extractTtsTags()`: Uses `indexOf` to scan through text section by section, extracting all content within `<tts>...</tts>` tags. Choosing `indexOf` over regex is to avoid boundary issues with special characters and nested pattern matching, improving parsing stability
2. **Deduplication Mechanism**: Through `sentTtsTexts` (`Map<string, Set<string>>`), records texts already played for each session; the same text will not be repeatedly played in the session, avoiding voice redundancy

Code Detail Description:

- `indexOf(openTag, start)`: Searches for the `<tts>` opening tag starting from the `start` position,

returning the tag's starting index, or -1 if not found
- `indexOf(closeTag, openIdx + openTag.length)`: Searches for the `</tts>` closing tag after the opening tag, ensuring correct matching of the nearest closing position
- `content.slice(openIdx + openTag.length, closeIdx).trim()`: Extracts the text content between opening and closing tags and trims leading/trailing whitespace
- `start = closeIdx + closeTag.length`: Updates the search starting position to after the closing tag, supporting multiple `<tts>` tags within the same text
- `sentTtsTexts`: `Map<sessionId, Set<text>>` structure, using session ID as key and played text set as value, implementing session-level deduplication

5.3 TTS Server

```
~/yahboomcar_ros2_ws/dofbot_pro_ws/src/ackermann_mcp/ackermann_mcp/openclaw_bridge.py
```

- `init_tts_engine`: Initialize TTS speech synthesis engine
    - When `USE_OLINE_TTS` is `False`: Load local `PiperTTS` engine for offline speech synthesis
    - When `tts_supplier` is `aliyun`: Load `TongyiTTS` (Alibaba Cloud Tongyi), requires API Key, model, and voice tone configuration
    - When `tts_supplier` is `xunfei`: Load `XunfeiTTS` (iFlytek), requires key path and encryption key configuration
- `_process_tts_queue`: TTS queue processing worker thread
    - Continuously checks the stop flag and retrieves text to synthesize from the queue (0.5s timeout)
    - Calls `_handle_tts` to process each text
- `_handle_tts`: Handle single TTS request
    - Wait for `play_audio` Action server to be ready
    - Call `tts_engine.synthesize` to synthesize text into an audio file
    - Send `PlayAudio` Action goal, handle goal acceptance/rejection via callback
    - Wait for playback to complete, check stop flag during playback, support canceling ongoing playback

Code Detail Description:

- `self.USE_OLINE_TTS`: Offline TTS switch. When `False`, load local PiperTTS engine; when `True`, load online engine based on `tts_supplier`
- `self.tts_supplier`: Online TTS provider selection, supports `aliyun` (Alibaba Cloud Tongyi) and `xunfei` (iFlytek)
- `self.tts_engine.synthesize(text, self.tts_out_path)`: Call TTS engine to synthesize text into an audio file, saved to `tts_out_path` path
- `self.audio_client.wait_for_server(timeout_sec=3.0)`: Wait for `play_audio` Action server to come

online; skip this playback if timeout exceeds 3 seconds

- `PlayAudio.Goal()`: Construct Action goal message for playing audio, `file_path` specifies the synthesized audio file path
- `_goal_response_cb`: Action goal sending callback, checks if the server accepted the goal; if rejected, logs a warning and ends
- `_tts_goal_handle_lock`: Thread lock, protects concurrent access to `_current_tts_goal_handle`, ensuring safety for cancellation operations
- `_result_cb`: Action completion callback, clears current goal handle and sets `done_event` to notify main thread
- `done_event.wait(timeout=0.1)`: Main loop polls playback completion status at 0.1-second intervals, checks stop flag during polling to support playback cancellation
- `gh.cancel_goal_async()`: Asynchronously cancel the currently playing Action goal when the stop flag is detected

```
    def init_tts_engine(self):
        if not self.USE_OLINE_TTS:
            from multi_brains_pre.utils.tts import  PiperTTS
            self.tts_engine=PiperTTS(self.language)
            if self.tts_engine.init_tts_engine() :
                self.get_logger().info(Fore.GREEN+"Piper TTS_Engine initialized
successfully"+Fore.RESET)
            else:
                error_msg="Piper TTS_Engine initialized failed"

                self.get_logger().error(Fore.RED+error_msg+Fore.RESET)
                raise RuntimeError(error_msg)
            return

        if self.tts_supplier=="aliyun":
            from multi_brains_pre.utils.tts import TongyiTTS

 self.tts_engine=TongyiTTS(self.aliyun_api_key,self.tts_model,self.tts_voice_tone)
            result=self.tts_engine.init_tts_engine()
            if result[0]:
                self.get_logger().info(Fore.GREEN+"Tongyi TTS_Engine initialized
successfully"+Fore.RESET)
            else:
                self.get_logger().error(Fore.RED+f"Tongyi TTS_Engine initialized failed:
{result[1]}"+Fore.RESET)

                raise RuntimeError(f"Tongyi TTS_Engine initialized failed:{result[1]}")


        elif self.tts_supplier=="xunfei":
            from multi_brains_pre.utils.tts import  XunfeiTTS

 self.tts_engine=XunfeiTTS(self.key_path,self.xunfei_encrypted_key,self.config_file)
```

```
            if self.tts_engine.init_tts_engine():
                self.get_logger().info(Fore.GREEN+"Xunfei TTS_Engine initialized
successfully"+Fore.RESET)
            else:

                error_msg="Xunfei TTS_Engine initialized failed"
                self.get_logger().error(Fore.RED+error_msg+Fore.RESET)
                raise RuntimeError(error_msg)
    def _process_tts_queue(self):
        """TTS queue processing worker thread"""
        while not self._tts_worker_stop_event.is_set():
            try:
                text = self._tts_queue.get(timeout=0.5)
            except queue.Empty:
                continue
            self._handle_tts(text)

    def _handle_tts(self, text: str):
        if not self.audio_client.wait_for_server(timeout_sec=3.0):
            self.get_logger().warn(Fore.YELLOW + "play_audio action server not available,
skip voice" + Fore.RESET)
            return
        if not self.tts_engine.synthesize(text, self.tts_out_path):
            self.get_logger().error(Fore.RED + "Speech synthesis failed. Check whether the
TTS model is available" + Fore.RESET)
            return

        goal = PlayAudio.Goal()
        goal.file_path = self.tts_out_path

        done_event = threading.Event()

        def _goal_response_cb(future):
            goal_handle = future.result()
            if not goal_handle or not goal_handle.accepted:
                self.get_logger().warn("TTS play goal rejected")
                done_event.set()
                return
            with self._tts_goal_handle_lock:
                if self._tts_worker_stop_event.is_set():
                    goal_handle.cancel_goal_async()
                    done_event.set()
                    return
                self._current_tts_goal_handle = goal_handle
            result_future = goal_handle.get_result_async()
            result_future.add_done_callback(_result_cb)

        def _result_cb(future):
            with self._tts_goal_handle_lock:
                self._current_tts_goal_handle = None
            done_event.set()

        send_future = self.audio_client.send_goal_async(goal)
```

```
        send_future.add_done_callback(_goal_response_cb)

        while not done_event.wait(timeout=0.1):
            if self._tts_worker_stop_event.is_set():
                with self._tts_goal_handle_lock:
                    gh = self._current_tts_goal_handle
                if gh is not None:
                    gh.cancel_goal_async()
                break
```